

Drizzle ORM x PostgreSQL



A friendly one-page guide for the type-safe SQL ORM that thinks in tables. Setup → schema → migrate → ship.

TypeScript-first

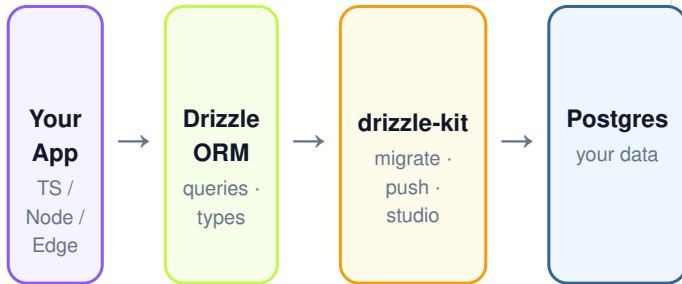
Zero deps

SQL-like

Postgres / MySQL / SQLite

Edge ready

1 The Big Picture



Drizzle is the runtime query builder. drizzle-kit is the CLI that handles schema, migrations, and the visual Studio.

2 Install in 30 seconds

```
# Pick a Postgres driver
npm i drizzle-orm postgres
npm i -D drizzle-kit tsx

# Or with node-postgres
npm i drizzle-orm pg
npm i -D drizzle-kit @types/pg tsx

# Edge / serverless: Neon HTTP
npm i drizzle-orm @neondatabase/serverless
```

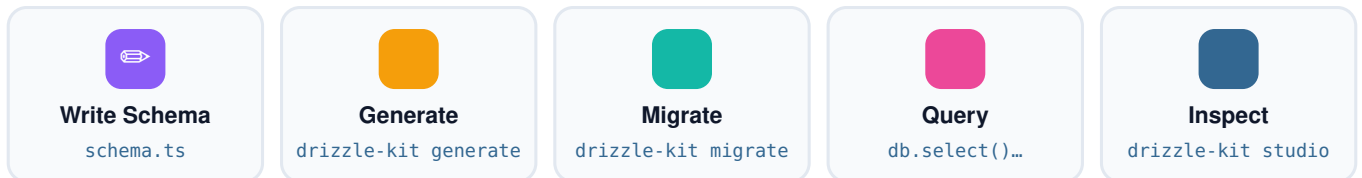
postgres-js

node-postgres

@neondatabase/serverless

@vercel/postgres

3 The Drizzle Workflow — five happy steps



Prototyping fast? Skip `generate + migrate` and use `drizzle-kit push` to sync schema → DB directly.

4 Define a Schema

```
// db/schema.ts
import { pgTable, serial, text, integer,
  timestamp, boolean } from 'drizzle-orm/pg-core';

export const users = pgTable('users', {
  id: serial('id').primaryKey(),
  email: text('email').notNull().unique(),
  name: text('name'),
  active: boolean('active').default(true),
  createdAt: timestamp('created_at').defaultNow(),
});

export const posts = pgTable('posts', {
  id: serial('id').primaryKey(),
  title: text('title').notNull(),
  authorId: integer('author_id')
    .references(() => users.id),
});
```

5 Connect to Postgres

```
// db/index.ts
import { drizzle } from 'drizzle-orm/postgres-js';
import postgres from 'postgres';
import * as schema from './schema';

const client =
  postgres(process.env.DATABASE_URL!);
export const db = drizzle(client, { schema });
```

DRIZZLE.CONFIG.TS

```
import { defineConfig } from 'drizzle-kit';

export default defineConfig({
  schema: './db/schema.ts',
  out: './drizzle',
  dialect: 'postgresql',
  dbCredentials: { url:
    process.env.DATABASE_URL! },
});
```

6 drizzle-kit Commands — your toolbox

COMMAND	WHAT IT DOES	WHEN TO USE IT
drizzle-kit generate	Differs schema vs. snapshot → writes a new SQL migration file in <code>/drizzle</code> .	After every schema change you want to commit to git.
drizzle-kit migrate	Runs pending SQL migrations against your DB and updates the journal.	Production deploys & teammates pulling new code.
drizzle-kit push	Pushes schema changes <i>directly</i> to the DB — no migration files.	Rapid prototyping, hackathons, throwaway DBs. Avoid in prod.
drizzle-kit pull	Introspects an existing DB and generates a Drizzle schema file.	Adopting Drizzle on a legacy database.
drizzle-kit studio	Launches a local web UI at https://local.drizzle.studio to browse + edit data.	Daily dev work — like a beautiful pgAdmin.
drizzle-kit check	Verifies your migration history has no collisions or missing files.	CI step before merging schema changes.
drizzle-kit drop	Removes a migration from the journal (file kept).	Undoing a generated migration before it ships.
drizzle-kit up	Upgrades old snapshots to the latest drizzle-kit format.	After bumping <code>drizzle-kit</code> versions.

Pro tip: Add scripts to `package.json` → `"db:generate": "drizzle-kit generate", "db:migrate": "tsx ./db/migrate.ts", "db:studio": "drizzle-kit studio"`.

7 CRUD in One Glance

SELECT

```
const all = await db.select().from(users);

const one = await db.select()
  .from(users)
  .where(eq(users.id, 1))
  .limit(1);
```

INSERT

```
const [u] = await db.insert(users)
  .values({ email: 'a@b.io', name: 'Ada' })
  .returning();
```

UPDATE

```
await db.update(users)
  .set({ active: false })
  .where(eq(users.id, u.id));
```

DELETE

```
await db.delete(users)
  .where(eq(users.id, u.id));
```

9 Generate & Migrate, in practice

```
# 1. Edit db/schema.ts
# 2. Generate SQL migration
npx drizzle-kit generate

# 3. Apply it (option A: CLI)
npx drizzle-kit migrate

# 3. Apply it (option B: in code)
// db/migrate.ts
import { migrate } from
  'drizzle-orm/postgres-js/migrator';
await migrate(db, { migrationsFolder: './
  drizzle' });
```

⚠ **push vs migrate:** `push` mutates the DB without a paper trail. Use `generate + migrate` for anything that ships.

8 Joins, Filters & Helpers

```
import { eq, and, or, gt, like, desc, sql }
  from 'drizzle-orm';

// inner join + filter + order
const rows = await db
  .select({
    title: posts.title,
    author: users.name,
  })
  .from(posts)
  .innerJoin(users, eq(posts.authorId,
    users.id))
  .where(and(
    eq(users.active, true),
    like(posts.title, '%drizzle%')
  ))
  .orderBy(desc(posts.id))
  .limit(10);

// raw SQL when you need it
const count = await db.execute(
  sql`select count(*) from ${users}`);
```

RELATIONAL QUERY API

```
const withPosts = await
  db.query.users.findMany({
    with: { posts: true },
    where: (u, { eq }) => eq(u.active, true),
  });
```

10 Drizzle Studio

A local web app to browse tables, edit rows, and run queries — like a designer-grade pgAdmin.

```
npx drizzle-kit studio
# → https://local.drizzle.studio

# custom port / host
npx drizzle-kit studio --port 4000 --host
0.0.0.0

# verbose logging when debugging
npx drizzle-kit studio --verbose
```

- Reads connection from `drizzle.config.ts`
- Inline edit + filter + sort, like a spreadsheet
- Shows relations defined in your schema

11 Relations at a glance

```
users ————— id serial PK | 1 : N
email text UNIQUE | name text ▼ posts id serial
PK title text author_id integer FK
```

```
import { relations } from 'drizzle-orm';

export const usersRelations = relations(
  users, ({ many }) => ({ posts: many(posts) })
);
export const postsRelations = relations(
  posts, ({ one }) => ({
    author: one(users, {
      fields: [posts.authorId],
      references: [users.id],
    }),
  })
);
```

12 Power-ups & Patterns

TRANSACTIONS

```
await db.transaction(async (tx) => {
  await tx.insert(users).values({...});
  await tx.insert(posts).values({...});
});
```

PREPARED STATEMENTS

```
const byId = db.select().from(users)
  .where(eq(users.id, sql.placeholder('id')))
  .prepare('byId');
await byId.execute({ id: 42 });
```

INFERRED TYPES — FOR FREE

```
type User = typeof users.$inferSelect;
type NewUser = typeof users.$inferInsert;
```

Remember: Drizzle queries are SQL-shaped, so if you can think it in SQL, you can write it in Drizzle.

13 Quick Troubleshooting

- **“No config path provided”** → add `drizzle.config.ts` at project root.
- **Migrations not applying** → check `out` folder matches `migrationsFolder`.
- **Type errors after schema edit** → re-run `drizzle-kit generate` & restart TS server.
- **Studio shows no tables** → confirm `DATABASE_URL` & that you've actually run a migration.
- **SSL on Neon / Supabase** → use the connection string they give you; it already includes `sslmode=require`.

14 Mini-Glossary

- **Schema** — your TS file describing tables & columns.
- **Snapshot** — JSON record of schema state used to diff.
- **Migration** — SQL file applied in order to evolve the DB.
- **Journal** — list of applied migrations stored in DB & `/drizzle`.
- **Dialect** — which SQL flavor (`postgresql`, `mysql`, `sqlite`).
- **Driver** — the lib that actually talks to the DB (`postgres-js`, `pg`, `neon`).

Made with ♥ for fast, type-safe Postgres · drizzle-orm · drizzle-kit · postgres · *orm.drizzle.team*